

```
public int X { get; private set; }
public int Y { get; private set; }
// Note: we must invoke the default constructor of
a struct before
// assigning to its fields.
public Vec2D(int x, int y) : this()
{
    X = x;
    Y = y;
}
}
```

Like C++, a struct defines something that looks like a class. However, unlike C++, struct is not a synonym for class. A struct is treated like a primitive type in Java with respect to memory semantics. This comes with some restrictions.

3. structs cannot extend any other classes. They can, however, implement any number of interfaces. They also cannot be extended by any other structs or classes (i.e., they are automatically sealed).

4. structs automatically define their own default (no-parameter) constructor that initialises all fields to their default values. You cannot change this constructor.

5. Because structs are not references, they cannot be assigned null. However, sometimes you want to be able to assign null to a struct as a way of saying "nothing", e.g., an option type. To do so, you can mark a variable with the type of the struct as nullable.

```
public class NullableTest
{
    public void Test ()
    {
        // The "?" denotes v should really be
Nullable<Vec2D>
        Vec2D? v;
        v = new Vec2D(0, 0);
        v = null;
    }
}
```

Thus, structs are very useful for "plain old datatypes" , PODs, that are